

เครื่องปั้นดินเผา

นักโบราณคดีได้ค้นพบเครื่องปั้นดินเผาโบราณที่เหมือนกันทุกประการจำนวน k ใบ นักโบราณคดีต้องการทดสอบหาความสูงวิกฤติ (ความสูงที่เครื่องปั้นดินเผาเหล่านี้แตกแล้วแตก) โดยการปล่อยจากหน้าผาซึ่งมีจุดให้ยืนปล่อยได้ทั้งหมด n ระดับ โดยมีหมายเลขตั้งแต่ 1 ถึง n

เครื่องปั้นดินเผานี้จะมีความสูงวิกฤติ h ($1 \leq h \leq n$) อยู่หนึ่งค่า ซึ่งหากปล่อยเครื่องปั้นดินเผาจากระดับที่ต่ำกว่า h เครื่องปั้นดินเผาจะไม่แตกและสามารถนำมาทดสอบซ้ำได้แต่หากปล่อยจากระดับ h หรือสูงกว่านั้นเครื่องปั้นดินเผาจะแตกและไม่สามารถใช้งานได้อีกต่อไป ภารกิจของคุณคือการหาความสูงวิกฤติ h โดยปล่อยเครื่องปั้นดินเผานี้เป็นจำนวนครั้งที่น้อยที่สุด โดยถ้าเครื่องปั้นดินเผาแตก k ใบแล้วจะไม่สามารถปล่อยเพิ่มได้

รายละเอียดการเขียนโปรแกรม

คุณจะต้องเขียนฟังก์ชันต่อไปนี้:

```
int find_breaking_point(int n, int k);
```

- ฟังก์ชันนี้จะถูกเรียก **20,000** ครั้ง
- การเรียกแต่ละครั้งถือเป็นการทดลองที่เป็นอิสระต่อกัน
- n : จำนวนระดับความสูงทั้งหมดสำหรับการทดลองครั้งปัจจุบัน
- k : จำนวนเครื่องปั้นดินเผาที่คุณมีสำหรับการทดลองครั้งปัจจุบัน
- ฟังก์ชันของคุณจะต้องคืนความสูงวิกฤติ h ที่ถูกต้อง

ภายในฟังก์ชันของคุณ สามารถเรียกใช้ฟังก์ชันดังต่อไปนี้ได้:

```
bool drop(int height);
```

- $height$: ระดับความสูง (ตั้งแต่ 1 ถึง n) ที่คุณต้องการปล่อยเครื่องปั้นดินเผา
- ฟังก์ชันนี้จะคืนค่า `true` หากเครื่องปั้นดินเผาแตก และ `false` หากไม่แตก
- เครื่องปั้นดินเผาที่แตกแล้วจะไม่สามารถนำมาใช้ในการทดลองครั้งปัจจุบันได้อีก
- ถ้า $height$ น้อยกว่า 1 หรือ มากกว่า n เกรดเดอร์จะรายงานเป็น Wrong Answer (0 คะแนน)
- ถ้าเรียกใช้หลังเครื่องปั้นดินเผาแตกหมดแล้ว เกรดเดอร์จะรายงานเป็น Wrong Answer (0 คะแนน)

หมายเหตุ: ให้เขียน `bool drop(int height);` ไว้ด้านบนสุดของ submission

การให้คะแนน

ฟังก์ชัน `find_breaking_point` จะถูกเรียกทั้งหมด 20,000 ครั้ง

- 10,000 ครั้งใช้ Randomized grader
- 10,000 ครั้งใช้ Adaptive grader

คะแนนจะลดลงตามจำนวนครั้งที่เรียกใช้ drop ที่เพิ่มขึ้น

$score_i = \max(1, C \cdot \frac{usage_i}{threshold_i})$ ซึ่ง $threshold_i$ จำนวนครั้งที่ดีที่สุดสำหรับ worst-case scenario ในการทดลองครั้งที่ i

$$FinalScore = \min_{1 \leq i \leq 20\,000} (score_i)$$

การให้คะแนนแบ่งเป็น 2 ส่วน (50% + 50%):

1. ส่วน Randomized (50%)

- ตัวตรวจจะสุ่มเลือกค่าความสูงวิกฤติในแต่ละครั้ง
- หากฟังก์ชันของคุณคืนค่าความสูงวิกฤติผิดแม้เพียงครั้งเดียว คะแนนส่วนนี้จะเป็น 0
- ถ้าคำตอบถูกและไม่ผิดเงื่อนไขข้างต้นคะแนนจะไม่ลดไปต่ำกว่า 1

2. ส่วน Adaptive (50%)

- ตัวตรวจจะเลือกค่าความสูงวิกฤติตามลำดับโดยอ้างอิงจากผลการเรียก drop ก่อนหน้า
- หากฟังก์ชันของคุณคืนค่าความสูงวิกฤติผิดแม้เพียงครั้งเดียว คะแนนส่วนนี้จะเป็น 0
- หากจำนวนครั้งที่เรียก drop ไม่เพียงพอจนไม่สามารถสรุปหาความสูงวิกฤติได้ คะแนนส่วนนี้จะเป็น 0
- ถ้าคำตอบถูกและไม่ผิดเงื่อนไขข้างต้นคะแนนจะไม่ลดไปต่ำกว่า 1

หมายเหตุ: หากเรียก drop หลังจากเครื่องปั่นดินเผาทั้งหมดแตกแล้ว เกรดเดอร์จะรายงานเป็น Wrong Answer (0 คะแนน)

ขอบเขต

- $1 \leq k \leq n \leq 20000$

ปัญหาย่อย

1. (5 คะแนน) $k \geq \lceil \log_2(n) \rceil$
2. (20 คะแนน) $k = 2$
3. (7 คะแนน) $k = 3, n \leq 4\,000$
4. (8 คะแนน) $k = 4, n \leq 4\,000$
5. (10 คะแนน) $n \leq 100$
6. (40 คะแนน) $n \leq 4\,000$
7. (10 คะแนน) ไม่มีเงื่อนไขเพิ่มเติม

สำหรับปัญหาย่อยที่ 1 ถ้าไม่ได้คะแนนเต็ม 5 คะแนน จะได้ 0 คะแนน

ตัวอย่าง

เกรตเตอร์เรียก `find_breaking_point(10, 1)` และความสูงวิกฤติซ้อนอยู่คือ $h = 5$

1. เรียก `drop(1)` ได้ผลลัพธ์ `false`
2. เรียก `drop(2)` ได้ผลลัพธ์ `false`
3. เรียก `drop(3)` ได้ผลลัพธ์ `false`
4. เรียก `drop(4)` ได้ผลลัพธ์ `false`
5. เรียก `drop(5)` ได้ผลลัพธ์ `true`
6. `find_breaking_point(10, 1)` คืนค่า 5

ข้อจำกัด

- Time limit: 1 second
- Memory limit: 128 MB

Hints and Solutions

<https://docs.google.com/document/d/e/2PACX-1vQXbuVMf44E4r1yWe0Rh3PwzdhP2tKgncM5EKDp82vVi9kky6tZ5o1BHeqNV1IDojsGwQdEIYGTLFH/pub>

pots © 2025 by [Gan S.](#) is licensed under [CC BY 4.0](#)